

Aligning Requirement for Large Language Model's Code Generation

Zhao Tian

College of Intelligence and Computing, Tianjin University
Tianjin, China
tianzhao@tju.edu.cn

Junjie Chen*

College of Intelligence and Computing, Tianjin University
Tianjin, China
junjiechen@tju.edu.cn

Abstract

Code generation refers to the automatic generation of source code based on a given programming specification, which has garnered significant attention particularly with the advancement of large language models (LLMs). However, due to the inherent complexity of real-world problems, the LLM-generated code often fails to fully align with the provided specification. While state-of-the-art agent-based techniques have been proposed to enhance LLM code generation, they overlook the critical issue of specification perception, resulting in persistent misalignment issues. Given that accurate perception of programming specifications serves as the foundation of the LLM-based code generation paradigm, ensuring specification alignment is particularly crucial. In this work, we draw on software requirements engineering to propose *Specine*, a novel specification alignment technique for LLM code generation. Its key idea is to identify misaligned input specifications, lift LLM-perceived specifications, and align them to enhance the code generation performance of LLMs. Our comprehensive experiments on four state-of-the-art LLMs across five challenging competitive benchmarks by comparing with ten state-of-the-art baselines, demonstrate the effectiveness of *Specine*. For example, *Specine* outperforms the most effective baseline, achieving an average improvement of 29.60% across all subjects in terms of Pass@1.

CCS Concepts

• **Software and its engineering** → **Automatic programming**;
• **Computing methodologies** → *Natural language processing*;
Neural networks.

Keywords

Code Generation, Large Language Model, Agent, Requirements Engineering

ACM Reference Format:

Zhao Tian and Junjie Chen. 2026. Aligning Requirement for Large Language Model's Code Generation. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3744916.3764572>

*Junjie Chen is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. ICSE '26, Rio de Janeiro, Brazil

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2025-3/26/04

<https://doi.org/10.1145/3744916.3764572>

1 Introduction

Code generation refers to the automatic generation of source code from a given programming specification. In recent years, it has attracted significant attention from both academia and industry due to its potential to reduce repetitive programming tasks and improve software development productivity [16, 30, 44, 52]. Advancements in LLMs, such as DeepSeek-Coder [20] and Gemini [47], have led to substantial improvements in code generation [57]. However, LLMs still face significant challenges, particularly when handling complex requirements [38, 51, 59]. For example, even advanced GPT-4 [1] generates code passing all test cases for only 12.10% of real-world competitive programming problems [27]. These performance issues impede the practical application of LLMs, potentially compromising software quality [48, 50]. Therefore, enhancing the code generation performance of LLMs is critical.

Recently, various prompt-based [32, 41, 51] and agent-based [13, 24, 59] techniques have been proposed to enhance the code generation performance of LLMs. In general, these techniques share four main steps: (1) Perception, where the LLM is provided with a requirement specification and perceives its intent; (2) Planning, where the LLM employs diverse planning strategies (e.g., task decomposition or algorithm selection) based on its specification perception; (3) Implementation, where the LLM generates code based on the devised plan; (4) Repair, where the LLM leverages feedback (e.g., test execution messages) to fix incorrect code. Among them, agent-based code generation has emerged as the state-of-the-art [25, 34], which utilizes specialized LLM agents to simulate the human software development workflow. For example, Self-collaboration [13] enables LLMs to take on different roles (e.g., analyst, coder, and tester), each responsible for a specific sub-task in the development process. PairCoder [59], the state-of-the-art agent-based technique, employs a navigator agent to explore diverse solution plans and a driver agent to repair implementations using execution feedback.

However, existing LLM-based code generation techniques [13, 24, 32, 41] mainly focus on the latter three stages (i.e., planning, implementation, and repair), while overlooking a fundamental issue in the first stage: the perception bias of LLMs towards the original input specification. Specifically, LLMs may implicitly ignore or misperceive key ingredients within the input specification, leading to generated code that deviates from the intended requirement. *The difference between the original input specification and the actual LLM-perceived specification is referred to as specification misalignment.* Actually, inaccurate perception can limit the capability of optimizing the subsequent three stages in improving LLM-based code generation, since it may fundamentally produce inaccuracies in the three stages (such as inaccurate planning). In addition, software requirements engineering research [7, 36, 46, 58] emphasizes

that effective software development must be built upon an accurate perception of fundamental specifications. Therefore, within the current LLM-driven automated code generation paradigm, ensuring specification alignment is particularly critical. That is, addressing the specification misalignment in LLM-based code generation is a key direction.

In this paper, we propose *Specine* (Specification Alignment), a novel specification alignment technique to enhance the code generation performance of LLMs. Specifically, *Specine* automatically identifies the misaligned specification, lifts the LLM-perceived specification, and aligns it with the original input specification, thereby facilitating correct code generation. However, designing an effective specification alignment technique presents the following key challenges: (1) *How to identify the misaligned ones from input specifications?* Performing alignment on all cases can incur unnecessary overhead, even may misprocess the originally-correct cases. (2) *How to obtain the actual LLM-perceived specification for the misaligned one?* Once a misaligned specification is identified, explicitly extracting the actual LLM-perceived specification facilitates the subsequent specification alignment. (3) *How to align the LLM-perceived specification to generate correct code?* Designing comprehensive and effective alignment rules is essential for aligning the LLM-perceived specification with the original input specification, thereby enhancing code generation performance.

To address the first challenge, *Specine* implements a dual-agent component comprising a coder agent to generate the initial code and a tester agent to estimate its correctness. The correctness of the generated code serves as an indicator of the LLM's perception of the input specification [22, 54]. If the specification is identified as potentially misaligned, *Specine* activates subsequent components for specification alignment. To address the second challenge, *Specine* employs a novel specification lifting component that explicitly extracts the LLM-perceived specification for effective misalignment analysis. This LLM-perceived specification is lifted from the low-level generated code using a domain-specific language (DSL) of requirement specifications, providing a high-level standardized representation. To address the third challenge, *Specine* applies ten pre-defined alignment rules (derived from software requirements engineering [11, 18, 19]) to systematically align different ingredients of the misaligned specification. In this way, *Specine* generates an aligned specification that is accurately perceived by the LLM, ultimately enhancing its code generation performance.

We conduct extensive experiments to evaluate *Specine* on four advanced LLMs (i.e., DeepSeek-Coder-1.5 [21], Qwen2.5-Coder [26], GPT-4o-mini [42], and Gemini-1.5-Flash [47]) based on five challenging competitive program-solving benchmarks (i.e., APPS [23], APPS-Eval [12], CodeContests-Raw [33], CodeContests [33], and xCodeEval [31]). Our results demonstrate that *Specine* significantly outperforms all 10 popular or state-of-the-art baselines across all 20 subjects (4 LLMs $\times$ 5 benchmarks), demonstrating our idea for enhancing LLM-based code generation by specification alignment. For example, the average improvement of *Specine* over all 10 baselines is 29.60%~93.55% in terms of Pass@1 (measuring the ratio of programming problems for which the generated code passes all test cases) across all subjects. Particularly, on the APPS dataset, the best performance in terms of Pass@1 achieved by all the four LLMs with the 10 baselines is 55.67% (using the Gemini-1.5-Flash model with

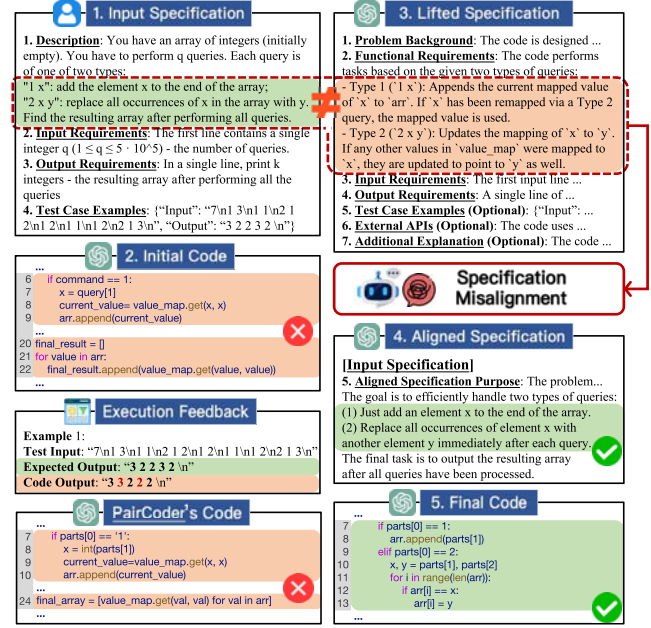


Figure 1: An example from Codeforces with GPT-4o-mini

the TGen technique), but *Specine* achieves 65.33% based on the same LLM. Also, we investigate the influence of the number of iterations, a key hyper-parameter in the agent-based framework. Our results show that as the number of iterations increases, *Specine* consistently outperforms all baselines. Furthermore, we construct five variants of *Specine* for an ablation study, confirming the contribution of each main component in *Specine*.

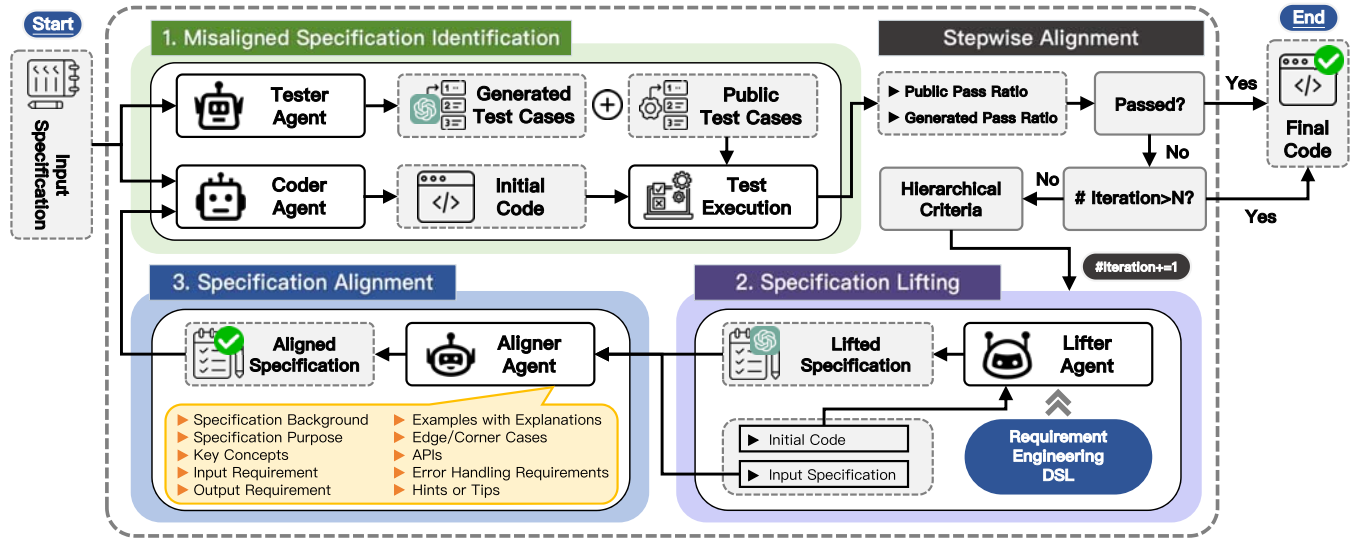
The main contributions of this paper are summarized as follows:

- **Novel Perspective:** We propose a novel perspective for enhancing the code generation performance of LLMs through the sophisticated specification alignment.
- **Tool Implementation:** We implement *Specine* following the novel perspective, which consists of identifying misaligned input specifications, lifting LLM-perceived specifications, and aligning them to generate the correct code.
- **Performance Evaluation:** We conduct extensive experiments on four advanced LLMs across five challenging benchmarks by comparing with ten state-of-the-art baselines, demonstrating the effectiveness of *Specine* in improving LLMs' code generation.
- **Data Availability:** We publicly release all accessible dataset and our source code at the project homepage [40] to facilitate the experiment replication, future research, and practical adoption.

2 Motivating Example

To illustrate the key idea of specification alignment, we present a real-world example to analyze the necessity of two key components in *Specine*: specification lifting (Section 3.2) and specification alignment (Section 3.3). The necessity of identifying misaligned specifications has been discussed in Section 1.

Figure 1 presents a real-world example from the Codeforces [9] platform. In this example, we employ an advanced LLM (GPT-4o-mini [42]) with Zero-shot learning [6] to generate the initial code based on the input specification. However, due to the complexity

Figure 2: Overview of *Specine*

of this problem, the generated code is incorrect. Specifically, the LLM fails to accurately perceive the “two query types” in the specification, leading to the corresponding error in the initial code. To further investigate this issue, we applied the state-of-the-art PairCoder [59] to adjust diverse solution plans and repair the code using execution feedback (shown in Figure 1). However, PairCoder’s code (shown in Figure 1) still exhibits the logical error. In contrast, by aligning the purpose behind the “two query types” (shown in the aligned specification of Figure 1), the LLM successfully generates a correct implementation (i.e., final code in Figure 1). *This motivates the potential of improving LLM-based code generation through specification alignment.*

Moreover, our natural attempt to directly instruct the LLM to align the specification is unsuccessful, still resulting in incorrect code. In contrast, explicitly extracting the LLM-perceived specification (i.e., lifted specification in Figure 1) enables the subsequent generation of correct code. This is because *Specine* systematically compares the lifted specification with the input specification, effectively analyzing specification misalignment for facilitating success alignment (i.e., the aligned specification in Figure 1). *This motivates the necessity of explicitly obtaining the LLM-perceived specification.*

Although the specification lifting component effectively helps analyze specification misalignment, mitigating this misalignment remains a significant challenge. To explore potential solutions, we further instruct the LLM to generate ten candidate aligned specifications based on the lifted specification (as evaluated in the *Specine_{woAR}* variant in RQ3). However, none of the generated specifications successfully address the misalignment, and thus the generated code remains incorrect. *This motivates the necessity of designing effective alignment rules for specification alignment, which can finally enhance LLM-based code generation.*

3 Approach

In this paper, we present *Specine*, a novel specification alignment technique designed to enhance the code generation performance of LLMs. Specifically, *Specine* automatically identifies misaligned

specifications, lifts LLM-perceived specifications, and aligns them to generate correct code. Figure 2 provides an overview of *Specine*, consisting of three main components: (1) **Misaligned Specification Identification** (Section 3.1) employs a coder agent to generate initial code and a tester agent to estimate its correctness to determine whether the LLM has correctly perceived the input specification. (2) **Specification Lifting** (Section 3.2) employs a novel LLM-driven lifting strategy to explicitly extract high-level perceived specifications from low-level generated code based on a pre-defined requirement DSL. (3) **Specification Alignment** (Section 3.3) applies ten pre-defined alignment rules to systematically align different ingredients of the misaligned specification, facilitating the subsequent code generation. In the following, we provide a detailed description of each component in *Specine*. Here, we reuse the real-world example introduced in Section 2 for illustration.

3.1 Misaligned Specification Identification

In *Specine*, identifying misaligned specifications is a critical step that evaluates whether the LLM correctly perceives the input specification. It is important to emphasize that the absence of this component would cause *Specine* to indiscriminately perform specification alignment on all input specifications. This indiscriminate processing could introduce significant drawbacks. First, for correctly LLM-perceived input specifications, forced alignment may lead to erroneous modifications due to the inherent hallucination issues of LLMs, potentially transforming originally correct code into incorrect implementations. Second, such redundant alignment operations impose unnecessary computational overhead, significantly increasing both time and token consumption.

As shown in Figure 2, we design a dual-agent framework comprising a coder agent to generate the initial code (shown in Figure 1) and a tester agent to generate test cases. The correctness of the generated code serves as an indicator of the LLM’s perception of the input specification [22, 54]. In particular, the tester agent independently generates test cases based on the input specification, without relying on the initial code. This independence ensures the

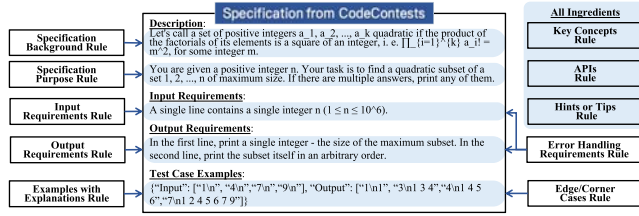


Figure 3: Correspondence between alignment rules and key specification ingredients

objectivity of test cases generation, mitigating any potential bias introduced by erroneous initial code. Following the existing research [25], we carefully design a prompt to guide the tester agent in generating effective test cases. The prompt details are as follows:

[Input Specification]

Instruction: Implement a representative set of test cases for the above specification, ensure that the generated test cases are correct:

- (1) **Basic Test Cases:** To verify the fundamental functionality under normal conditions.
- (2) **Edge Test Cases:** To evaluate the function's behavior under extreme or unusual conditions.
- (3) **Large-Scale Test Cases:** To assess the function's scalability with large data samples.

Please provide additional test cases in a json format: ``json {"inputs": [], "outputs": []}``

In practice, programming specifications commonly include public test cases. To comprehensively assess the generated code, *Specine* combines these public test cases with the additional test cases generated by the tester agent. These test cases can help ensure the reliability of this step for identifying misaligned specification. In Section 5.3, we further explore the influence of public test cases and generated test cases on the effectiveness of *Specine*, respectively. Upon executing these test cases, *Specine* estimates the correctness of the initial code. If the code fails to pass all test cases, the input specification is deemed misaligned, activating the subsequent components for specification alignment. Conversely, if the code successfully passes all test cases, it is considered the final output of *Specine*. We acknowledge that the LLM-generated test cases may contain partial inaccuracies and cannot guarantee complete correctness. Nevertheless, consistent with findings of prior studies [5, 34], they still contribute to enhancing the overall performance of LLM code generation. A systematic discussion of this phenomenon will be presented in Section 6.3.

3.2 Specification Lifting

After identifying misaligned input specifications based on the estimated correctness of the generated code, it is essential to thoroughly analyze the detailed misalignment by comparing the generated code and the input specification, which can facilitate specification alignment (as discussed in Section 2). However, directly comparing and analyzing the specification misalignment is inherently challenging due to the significant difference in abstraction levels between the initial code (i.e., low-level programming language) and the input specification (i.e., high-level requirement description). To effectively bridge this semantic gap, we design a specification lifting component that explicitly extracts the LLM-perceived specification from the generated code.

Inspired by existing lifting research [3, 39, 61], we propose a novel lifter agent (illustrated in Figure 2) that lifts the generated code into a DSL tailored for requirement specifications. Compared

to ordinary natural language descriptions, a requirement DSL provides a standardized representation of critical software requirements, mitigating common issues such as ambiguity and incompleteness. Building on research in software requirements engineering [3, 39, 61], we define a tailored requirement DSL for our task, encompassing the following key attributes:

```
{
  "Problem Background": "Describes the background and context of the code, providing the necessary background knowledge for understanding the code's functionality.",
  "Functional Requirements": "Summarizes the core functionality of the code, specifying its specific objectives or tasks, such as the problem being solved or the task being performed.",
  "Input Requirements": "Details the inputs required by the code, including input types, formats, and any constraints.",
  "Output Requirements": "Specifies the outputs of the code, including output types, formats, and any constraints.",
  "Test Case Examples": "Extracts the test cases included in the code, which are often used to verify code correctness and serve as usage examples.",
  "External APIs": "Lists any external APIs or library functions used by the code and describes their purpose and interactions.",
  "Additional Explanation": "Provides supplementary information, such as design intentions, potential limitations, or special considerations."
}
```

This highly standardized requirement DSL aligns with *IEEE specification standard* [11], ensuring that the extracted lifted specification (shown in Figure 1) comprehensively and effectively represents the LLM-perceived specification from the initial code. Furthermore, we discuss the scalability of the DSL design in Section 6.4, highlighting its potential to further enhance the generalizability and applicability of our approach.

Moreover, we design a prompt for the lifter agent that consists of the initial code, the definition of requirement DSL, and a task instruction ("Analyze the initial code exclusively and translate it as the lifted specification based on the defined requirement DSL."). Through LLM-driven specification lifting, the lifter agent generates the requirement DSL and parses it into a structured lifted specification template, thereby producing the final lifted specification. Particularly, our ablation results (in Section 5.3) demonstrate that the specification lifting strategy significantly outperforms the commonly-used test execution feedback strategy in existing agent-based code generation techniques.

3.3 Specification Alignment

The core idea of this component is to analyze potential misaligned ingredients between the LLM-perceived specification (i.e., the lifted specification) and the input specification, and then mitigate these misalignment using pre-defined alignment rules. Based on the existing studies in software requirements engineering [11, 18, 19], we define ten alignment rules that systematically align different kinds of ingredients in the misaligned specification (shown in Figure 3). These alignment rules are detailed as follows:

- **Specification Background:** further explains the background, motivation, or domain-specific knowledge required by the specification. This helps the LLM fully perceive the problem context for more accurate code generation.
- **Specification Purpose:** emphasizes the detailed objective or core tasks of the specification. This helps the LLM maintain focus on the intended goal, reducing the likelihood of generating code that deviates from the required functionality.
- **Key Concepts:** identifies and explains the critical terminology in the specification, ensuring that the LLM accurately perceives

the core terms and concepts, thereby minimizing code errors caused by conceptual misperception.

- **Input Requirements:** emphasizes the input requirements of the specification, including data types, formats, and constraints (e.g., value ranges, size limits, or specific conditions). It helps the generated code correctly process input data.
- **Output Requirements:** emphasizes the output requirements of the specification, including data types, formats, and constraints (e.g., precision, delimiters, or sorting rules). It helps the generated code correctly produce the expected output.
- **Examples with Explanations:** provides step-by-step analysis of test cases, illustrating the detailed logic from input to output. This enhances the LLM's perception of the programming logic.
- **Edge/Corner Cases:** introduces three additional LLM-generated test cases covering extreme or boundary conditions to ensure that the generated code handles unusual edge cases correctly.
- **APIs:** specifies external APIs or library functions relevant to the task, including their names and functionalities. This helps the LLM implement the required functions more effectively.
- **Error Handling Requirements:** defines the expected behavior when encountering invalid inputs, such as returning default values, raising exceptions, or applying special mechanisms. It ensures the code behaves as expected in exceptional circumstances.
- **Hints or Tips:** provides optional implementation suggestions, such as recommended algorithms or data structures. It serves as a supplementary guideline for addressing any remaining specification misalignment not covered by the preceding nine rules.

Furthermore, *Specine* employs an LLM-based aligner agent to facilitate specification alignment. Specifically, we design a prompt for the aligner agent, which consists of the input specification, the lifted specification, the ten pre-defined alignment rules, and a task instruction ("Analyze the input specification and the lifted specification to identify misalignment or omissions. Select one or more ingredients from the ten alignment rules to improve the input specification."). The aligner agent automatically generates the aligned ingredients for the input specification, which collectively constitute the aligned specification (as shown in Figure 1).

In particular, the specification alignment is implemented as a stepwise search process. In each iteration, *Specine* produces an aligned specification and subsequently uses it to generate new code. To measure the quality of different versions of generated code, we design a hierarchical criteria based on two pass ratios: (1) the primary pass ratio, which measures the pass ratio of public test cases provided in the input specification, and (2) the secondary pass ratio, which measures the pass ratio of LLM-generated test cases. When comparing different code, the primary pass ratio is considered first; only if this ratio is identical is the secondary pass ratio considered. This hierarchical criterion prioritizes the primary pass ratio, as the correctness of public test cases is guaranteed, whereas LLM-generated test cases may contain errors. Importantly, if the new code generated from the current aligned specification achieves a higher hierarchical pass ratio than the previous iteration (even if it does not fully pass all test cases), *Specine* retains the current aligned ingredients and proceeds to the next iteration. This greedy

optimization process ensures stepwise improvements during specification alignment process. In the future, more sophisticated search algorithms could be integrated to further enhance this component.

Besides, *Specine* terminates the alignment process once a pre-defined number of iterations (denoted as N) is reached, outputting the code with the highest hierarchical pass ratio. The number of iterations is a critical hyper-parameter for all agent-based techniques, and its impact will be discussed in Section 5.2. Through this specification alignment component, *Specine* can effectively achieve specification alignment for generating correct code.

4 Evaluation Design

Our study aims to address the following research questions (RQs):

- **RQ1:** How does *Specine* perform in terms of effectiveness and efficiency compared to the state-of-the-art techniques?
- **RQ2:** How do hyper-parameters affect *Specine*'s effectiveness?
- **RQ3:** How does each main component in *Specine* contribute to the overall effectiveness?

4.1 Benchmarks

To comprehensively evaluate *Specine*, we utilize five challenging benchmarks in our study: APPS [23], APPS-Eval [12], CodeContests-Raw [33], CodeContests [33], and xCodeEval [31]. These datasets are commonly used in many existing LLM-based code generation studies [27, 28, 51]. Notably, we do not include basic programming datasets such as HumanEval [6] and MBPP [4], as LLMs already achieve a Pass@1 exceeding 95% on these datasets with simple Zero-shot [15] setting. Instead, we select more challenging competition-level datasets to better assess the performance of *Specine*.

APPS consists of programming problems collected from various competitive programming platforms (e.g., LeetCode [45]), including 5,000 training data and 5,000 test data. To balance the evaluation cost and conclusion generalizability, we randomly sample 300 problems from test set based on the difficulty distribution, following the existing work [41, 51]. Additionally, to improve the comprehensiveness of the evaluation, APPS is extended into *APPS-Eval* by constructing over 100 additional test cases for each problem.

CodeContests-Raw is proposed by Google DeepMind [10], consisting of 13,328 training data, 117 validation data, and 165 test data. It is designed to evaluate the ability of LLMs to solve challenging programming problems. We also use the extended version, *CodeContests*, which includes ~190 additional test cases over the raw version. Following existing studies [5, 62], we use all 165 test data from both CodeContests-Raw and CodeContests.

xCodeEval is a competition-level multi-task benchmark, consisting of ~7,500 programming problems and 25 million code examples collected from Codeforces [9]. To maintain consistency with APPS, we randomly sample 300 problems from the code generation test data based on the frequency distribution of difficulty levels.

These datasets provide several public test cases (as part of the original specification), along with separate private test cases used for evaluating code correctness. Some studied baselines (including Self-repair [41], μ FiX [51], Self-collaboration [13], PairCoder [59], and TGen [37]) rely on the execution result of public test cases. However, in APPS, public test cases are presented in natural language within the original specification, rendering them non-executable.

Hence, for APPS, we randomly sample three test cases from private test cases to serve as executable public test cases. To prevent result inflation due to potential test case leakage, we remove the three cases from private test cases for performance evaluation.

4.2 Metrics

Following existing studies [29, 51], we use *Pass@k* and *AvgPassRatio* to evaluate the effectiveness of *Specine*. *Pass@k* measures the functional correctness of the generated code. For a given programming problem, the LLM generates k code instances. If any of the instances passes all the private test cases, the problem is considered solved. *Pass@k* is the percentage of problems solved out of the total number of problems. As demonstrated by existing studies [8, 13, 38], developers tend to consider the first code instance generated by the LLM, and thus we set $k = 1$ following existing work [34, 51, 59]. Note that *Pass@1* is a stricter criterion, making improvements in this metric both challenging and practically meaningful. *AvgPassRatio* measures the degree of correctness of the generated code on private test cases, and differs from *Pass@k* that evaluates whether the generated code is completely correct on private test cases. *AvgPassRatio* calculates the ratio of passing private test cases for each problem, and then computes the average ratio across all problems. Both metrics are largely complementary, higher *Pass@k* and *AvgPassRatio* values indicate better effectiveness.

Additionally, we evaluate the efficiency of *Specine* by measuring both *time overhead* and *token overhead* (which includes prompt token cost and generated token cost) following existing work [13, 24, 51]. Smaller values of time and token overhead indicate better efficiency in code generation.

4.3 Compared Techniques

To thoroughly evaluate *Specine*, we compare it with six representative or state-of-the-art agent-based techniques: **Self-collaboration** [13], **MetaGPT** [24], **AgentCoder** [25], **TGen** [37], **FlowGen** [34], and **PairCoder** [59]. They share the same high-level insight, which simulates the software development process by designing diverse agents. The primary difference among these techniques lies in the roles assigned to agents and the distinct strategies to address corresponding sub-tasks. Although our study focuses on agent-based techniques, for a more comprehensive evaluation of *Specine*, we also compare it with four typical or state-of-the-art prompt-based code generation techniques, including **Zero-shot** [6], **SCoT** [32], **Self-repair** [41], and **μ FiX** [51]. Due to space limitations, more details on these techniques can be found in their respective papers.

4.4 Implementation Details

To evaluate the performance of *Specine*, we select a variety of open-source and commercial LLMs for our study. For open-source LLMs, we chose two representative LLMs: Qwen-Coder [26] (version *qwen2.5-coder-7b-instruct*) and DeepSeek-Coder [21] (version *deepseek-coder-7b-instruct-v1.5*). Specifically, we download them via the Huggingface [56] platform and deploy them in a local environment for our experiments. For commercial LLMs, we select two widely-used and advanced LLMs: GPT-4o [42] (version *gpt-4o-mini-2024-07-18*) and Gemini-1.5 [47] (version *gemini-1.5-flash-002*). We access these commercial LLMs through the respective APIs

provided by OpenAI [43] and Google AI [2]. These LLMs, which have demonstrated excellent performance in code generation and are widely adopted [17, 53, 59], evaluate the generalizability of *Specine* to some extent. In our experiments, we set max tokens to 1024 and temperature to 0.8 for all LLMs. The maximum number of iterations N is set to 10 for *Specine* and baselines. As LLMs often generate code that includes natural language text segments (e.g., explanatory text), which can lead to compilation failures or execution errors. Following existing studies [35, 51], we utilize a code sanitizer tool [14] to preprocess the code generated by the LLMs. It can automatically detect and remove non-functional text segments from the generated code.

5 Results and Analysis

5.1 RQ1: Effectiveness and Efficiency

5.1.1 Process: To answer RQ1, we apply *Specine* and 10 compared techniques to four studied LLMs. We then evaluate the effectiveness of each technique across five widely-used benchmarks in terms of *Pass@1* and *AvgPassRatio*. In addition, we assess the efficiency of each technique in terms of time and token overhead.

5.1.2 Results: Table 1 shows the effectiveness and efficiency comparison results of these techniques. First, we observe that, on average, agent-based code generation techniques outperform prompt-based code generation techniques in terms of *Pass@1* and *AvgPassRatio*. This result confirms the effectiveness of the agent-based framework and further motivates the design of our *Specine*.

In particular, ***Specine* achieves the best effectiveness among all studied techniques, consistently demonstrating superior performance in both effectiveness metrics across all 20 subjects (4 LLMs $\times$ 5 benchmarks).** Specifically, *Specine* significantly improves all baselines by 29.60%~93.55% and 27.95%~79.12% in terms of *Pass@1* and *AvgPassRatio* on average across all subjects, respectively. Furthermore, the *Wilcoxon Signed-Rank Test* [55] at a significance level of 0.05 confirms that all p-values are smaller than 2.40×10^{-7} , demonstrating the statistically significant superiority of *Specine* over all baselines in terms of *Pass@1* and *AvgPassRatio*.

In addition, prompt-based code generation techniques outperform agent-based techniques in terms of both efficiency metrics (i.e., time and token overhead). However, considering the significant effectiveness of agent-based techniques, a certain degree of additional cost is justified, illustrating their excellent balance of cost and effectiveness. Notably, **our *Specine* achieves superior efficiency compared to all six agent-based baselines in terms of both efficiency metrics.** Specifically, *Specine* improves all agent-based baselines by 22.44%~39.14% and 9.69%~46.89% in terms of time and token overhead on average across all subjects, respectively. Furthermore, the *Wilcoxon Signed-Rank Test* [55] at a significance level of 0.05 confirms that all p-values are smaller than 1.56×10^{-2} , demonstrating the statistically significant superiority of *Specine* over all agent-based baselines in terms of time and token overhead.

5.2 RQ2: Influence of Hyper-parameter

5.2.1 Setup: The number of iterations is a key hyper-parameter in agent-based code generation techniques [25, 59]. In this research question, we investigate the influence of the number of iterations

Table 1: Effectiveness and efficiency comparison in terms of Pass@1 ($\uparrow$), AvgPassRatio ($\uparrow$), Time Overhead ($\downarrow$), and Token Overhead ($\downarrow$). APR is short for AvgPassRatio.

LLM	Technique	APPS		APPS-Eval		CodeContests-Raw		CodeContests		xCodeEval		Time (h)	Token (M)
		Pass@1	APR	Pass@1	APR	Pass@1	APR	Pass@1	APR	Pass@1	APR		
DeepSeek-Coder-v1.5	Zero-shot	18.00%	34.25%	8.33%	33.35%	7.27%	14.78%	2.42%	13.02%	11.00%	24.77%	3.50	0.64
	SCoT	19.00%	35.95%	8.67%	34.82%	7.27%	16.75%	3.64%	14.25%	12.33%	25.21%	5.15	2.43
	Self-repair	20.67%	38.38%	8.67%	36.23%	7.27%	17.33%	3.64%	14.73%	12.67%	26.65%	6.39	2.06
	μ FiX	23.00%	40.42%	10.67%	39.04%	9.70%	19.90%	4.24%	15.10%	13.33%	27.12%	18.94	6.29
	Self-collaboration	23.00%	35.37%	10.33%	32.84%	9.70%	18.48%	4.24%	15.96%	13.33%	24.83%	87.21	18.48
	MetaGPT	24.33%	38.27%	10.67%	34.84%	7.27%	13.79%	4.85%	12.52%	12.67%	20.88%	127.43	25.16
	AgentCoder	21.67%	37.53%	9.33%	33.59%	9.09%	16.01%	4.24%	14.74%	12.00%	25.34%	108.80	15.67
	TGen	22.33%	36.34%	10.00%	33.37%	7.88%	17.41%	3.03%	13.79%	15.00%	27.88%	73.44	17.36
	FlowGen	23.67%	41.09%	10.67%	38.86%	9.70%	15.87%	4.24%	14.36%	15.33%	29.30%	110.95	20.43
	PairCoder	23.67%	37.37%	9.67%	35.09%	10.30%	20.82%	4.24%	18.22%	13.33%	26.64%	75.99	16.93
	<i>Specine</i>	35.33%	59.62%	14.33%	50.99%	13.33%	28.67%	7.88%	30.16%	20.67%	45.58%	60.22	15.16
Qwen2.5-Coder	Zero-shot	19.67%	36.66%	9.33%	34.95%	7.88%	17.97%	5.45%	18.64%	10.67%	27.63%	3.23	0.54
	SCoT	21.67%	38.29%	10.00%	35.94%	8.48%	15.67%	6.06%	15.13%	11.33%	22.52%	6.62	2.37
	Self-repair	21.67%	37.12%	10.00%	36.05%	8.48%	19.82%	6.06%	19.93%	12.00%	24.17%	5.54	1.54
	μ FiX	24.33%	39.34%	11.00%	41.43%	10.91%	22.57%	6.67%	20.97%	13.33%	27.92%	19.99	6.08
	Self-collaboration	24.00%	35.88%	10.67%	33.24%	9.09%	20.82%	7.27%	22.21%	12.33%	27.08%	62.60	15.75
	MetaGPT	23.33%	36.04%	11.00%	33.78%	9.70%	18.17%	7.27%	19.38%	11.33%	23.92%	110.45	25.54
	AgentCoder	24.33%	38.06%	11.33%	33.76%	12.12%	22.36%	9.09%	22.52%	14.33%	28.27%	139.57	16.90
	TGen	26.33%	37.10%	11.00%	31.75%	9.09%	13.62%	6.06%	12.11%	14.33%	22.21%	74.42	17.11
	FlowGen	25.33%	38.61%	10.00%	34.60%	8.48%	15.60%	6.67%	14.87%	12.00%	24.72%	108.02	21.98
	PairCoder	30.33%	50.20%	11.67%	42.18%	13.94%	29.07%	9.09%	25.53%	16.00%	37.33%	84.81	15.64
	<i>Specine</i>	37.67%	60.63%	16.67%	51.53%	15.15%	32.44%	10.30%	34.11%	18.67%	42.09%	51.20	14.38
GPT-4o-mini	Zero-shot	33.67%	43.61%	13.33%	41.38%	8.48%	18.16%	4.85%	17.96%	23.67%	32.82%	1.67	0.61
	SCoT	35.33%	46.52%	14.33%	43.03%	11.52%	23.73%	6.06%	22.00%	25.00%	33.68%	1.43	2.46
	Self-repair	36.67%	47.01%	14.67%	43.24%	10.30%	19.94%	6.67%	20.66%	25.67%	34.88%	1.97	1.63
	μ FiX	40.33%	49.31%	15.67%	46.15%	12.12%	25.53%	9.09%	23.21%	28.67%	38.95%	5.62	6.55
	Self-collaboration	42.00%	55.19%	16.33%	48.25%	14.55%	27.04%	10.91%	25.72%	30.67%	45.48%	31.40	17.00
	MetaGPT	41.33%	49.90%	16.33%	45.83%	12.12%	25.69%	9.70%	25.78%	26.00%	35.14%	32.90	27.20
	AgentCoder	41.33%	49.16%	16.67%	44.21%	15.76%	26.58%	10.30%	26.48%	26.00%	33.46%	28.71	16.84
	TGen	41.00%	47.16%	16.67%	42.62%	9.09%	13.17%	7.88%	12.82%	30.67%	37.58%	48.21	18.32
	FlowGen	43.67%	52.32%	17.00%	45.43%	12.12%	21.43%	9.09%	19.35%	30.00%	38.00%	32.70	21.33
	PairCoder	49.00%	63.68%	19.67%	56.02%	18.79%	34.14%	11.52%	31.88%	31.67%	46.06%	29.87	16.71
	<i>Specine</i>	56.33%	72.98%	22.67%	63.55%	23.64%	40.65%	17.58%	42.10%	38.67%	57.34%	24.62	14.41
Gemini-1.5-Flash	Zero-shot	47.67%	59.33%	19.67%	51.65%	29.09%	39.30%	23.64%	33.49%	27.00%	41.44%	1.10	0.50
	SCoT	48.67%	59.51%	20.67%	51.10%	30.30%	40.08%	25.45%	35.05%	28.00%	41.26%	1.13	2.22
	Self-repair	49.33%	60.14%	21.33%	52.71%	32.33%	42.64%	25.45%	36.71%	29.33%	42.72%	1.50	0.99
	μ FiX	51.33%	62.05%	22.33%	56.69%	34.39%	44.22%	28.48%	40.93%	31.00%	44.10%	4.08	5.10
	Self-collaboration	54.33%	66.28%	22.67%	54.89%	36.97%	49.11%	30.30%	43.13%	31.67%	46.19%	21.74	10.71
	MetaGPT	55.00%	65.31%	22.67%	53.60%	35.76%	45.68%	28.48%	40.34%	29.00%	35.21%	21.74	22.79
	AgentCoder	53.33%	60.51%	22.33%	51.23%	42.42%	50.90%	32.73%	47.17%	30.67%	37.55%	20.22	13.83
	TGen	55.67%	68.96%	23.33%	58.76%	36.97%	50.06%	26.67%	42.82%	34.00%	47.12%	42.36	10.43
	FlowGen	53.33%	63.15%	21.67%	51.01%	40.00%	54.31%	31.52%	48.92%	29.67%	42.14%	22.22	13.89
	PairCoder	55.00%	68.47%	23.00%	56.82%	43.03%	54.42%	30.91%	47.05%	32.33%	46.72%	21.70	10.46
	<i>Specine</i>	65.33%	78.15%	29.33%	65.91%	44.85%	56.96%	36.36%	54.34%	38.67%	58.42%	17.61	9.77

(i.e., N) on the effectiveness of *Specine*. Specifically, for $1 \leq N \leq 10$, we evaluate the performance of *Specine* alongside six agent-based baselines across all 20 subjects (4 LLMs $\times$ 5 benchmarks) in terms of Pass@1 and AvgPassRatio.

5.2.2 Results: Figure 4 illustrates the variation in Pass@1 across different agent-based techniques as the number of iterations increases. Due to space limitations, we put the results for AvgPassRatio on our homepage [40]. First, we observe that as the number of iterations increases, all agent-based techniques exhibit improvements

in effectiveness for both Pass@1 and AvgPassRatio. However, across all iteration settings, *Specine* consistently outperforms all baselines, achieving 27.29%~42.86% higher Pass@1 and 25.86%~68.22% higher AvgPassRatio on average across all subjects. This demonstrates the stable superiority of *Specine* across different iteration settings. Furthermore, the *Wilcoxon Signed-Rank Test* [55] at a significance level of 0.05 confirms that all p-values are smaller than 1.09×10^{-4} , demonstrating the statistically significant superiority of *Specine* under varying iteration settings.

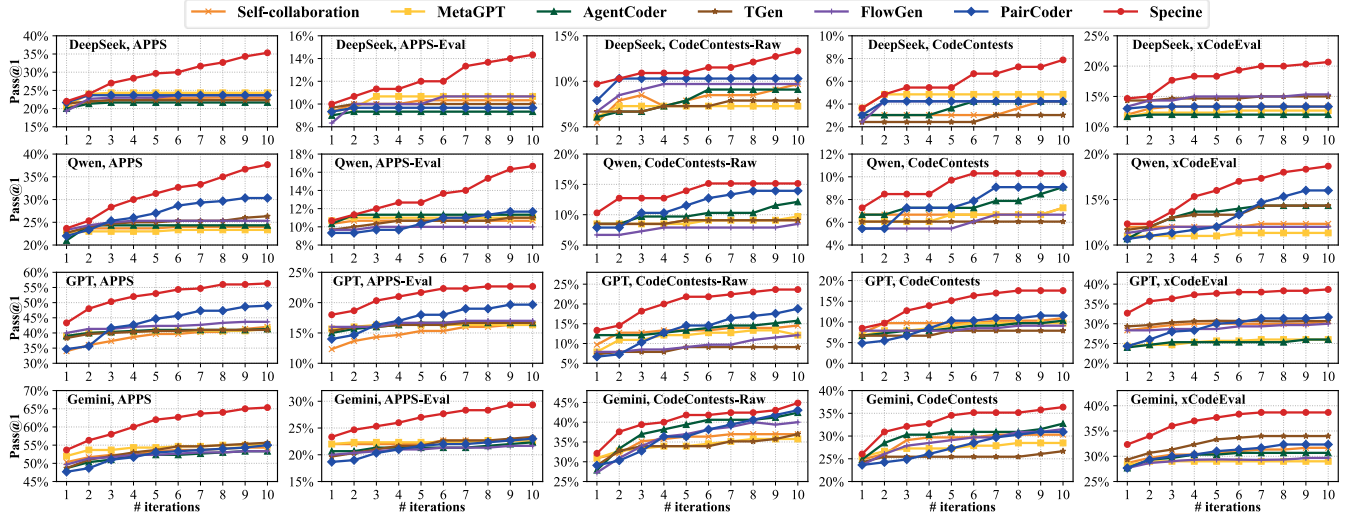


Figure 4: Influence of the number of iterations (N) in terms of Pass@1 (!)

In addition, the improvement achieved by increasing the number of iterations in *Specine* surpasses that of all baselines. Specifically, as N increases from 1 to 10, *Specine* achieves an average improvement of 47.97% in terms of Pass@1 across all subjects, whereas other baselines show an average improvement of 13.27%~45.40%. Similarly, for the AvgPassRatio metric, *Specine* demonstrates an average improvement of 37.02%, while other baselines achieve an average improvement of 9.97%~35.87%.

In particular, these baselines appear to reach an effectiveness plateau after a certain number of iterations, such as when $N \geq 3$ on APPS with DeepSeek-Coder. In contrast, *Specine* consistently exhibits an upward trajectory across all iterations on almost all subjects, demonstrating its ability to leverage additional iterations for continuous improvement in effectiveness. This is because *Specine*'s specification lifting component keeps improving alignment over time. Even when early iterations do not achieve perfect alignment, *Specine* can extract detailed information from newly generated code to progressively refine the LLM-perceived specification. In contrast, existing baselines rely primarily on coarse test feedback, which is less informative and may even lead to regression issues. We hypothesize that increasing the number of iterations may further enhance *Specine*'s performance; however, this entails a trade-off between effectiveness and computational cost.

5.3 RQ3: Contribution of Main Components

5.3.1 Variants: *Specine* consists of three main components: misaligned specification identification, specification lifting, and specification alignment. To investigate the contribution of each component, we construct five variants of *Specine* for evaluation.

For the misaligned specification identification component, we utilize both public test cases and LLM-generated test cases to assess the correctness of the initial code. To investigate separately the contribution of these two types of test cases, we construct two variants of *Specine*: namely *Specine*_{woPTC} and *Specine*_{woT}, where public test cases and the tester agent are removed, respectively.

For the specification lifting component, existing agent-based code generation techniques typically rely on the coarse-grained execution results of test cases as feedback to repair incorrect code. In contrast, *Specine* employs specification lifting to obtain finer-grained feedback messages. To assess the effectiveness of this component, we replace the specification lifting strategy with a test-execution feedback strategy (introduced in Self-repair [41]), resulting in a variant of *Specine* called *Specine*_{wTF}. Specifically, *Specine*_{wTF} utilizes test-execution feedback messages as input for the subsequent aligner agent to align the specification.

For the specification alignment component, we evaluate the contributions of both the aligner agent and pre-defined alignment rules by constructing two additional variants of *Specine*: *Specine*_{woA} and *Specine*_{woAR}. In *Specine*_{woA}, we remove the aligner agent and randomly select pre-defined alignment rules for specification alignment. In *Specine*_{woAR}, we eliminate the pre-defined alignment rules and instead provide a task instruction that guides the aligner agent to directly generate the aligned specification.

5.3.2 Results: Table 2 shows the comparison results between *Specine* and its five variants across all 20 subjects (4 LLMs $\times$ 5 benchmarks) in terms of Pass@1 and AvgPassRatio. Firstly, *Specine* consistently outperforms both *Specine*_{woPTC} and *Specine*_{woT} in terms of both metrics. On average, *Specine* improves 17.59% and 12.41% higher Pass@1 than *Specine*_{woPTC} and *Specine*_{woT}, and 25.12% and 11.84% higher AvgPassRatio, respectively. These results confirm the effectiveness of incorporating both public test cases and LLM-generated test cases in *Specine*. Additionally, *Specine*_{woT} demonstrates superior effectiveness over *Specine*_{woPTC}, achieving average improvements of 4.27% and 10.20%, respectively. This underscores the necessity of our hierarchical criteria (introduced in Section 3.3), which confirms that while public test cases are typically more reliable, LLM-generated test cases may introduce errors. Moreover, prior studies [37, 51, 59] have suggested that including public test cases within specifications is a common practice. However, *Specine*_{woPTC} addresses a more challenging scenario where no public test cases are available. The experimental results show that *Specine*_{woPTC} still

Table 2: Comparison between *Specine* and its variants in terms of Pass@1 ($\uparrow$) and AvgPassRatio ($\uparrow$). APR is short for AvgPassRatio.

LLM	Technique	APPS		APPS-Eval		CodeContests-Raw		CodeContests		xCodeEval	
		Pass@1	APR	Pass@1	APR	Pass@1	APR	Pass@1	APR	Pass@1	APR
DeepSeek-Coder-v1.5	<i>Specine</i> _{woPTC}	29.67%	46.70%	12.00%	41.41%	11.52%	22.10%	6.06%	18.89%	17.33%	32.52%
	<i>Specine</i> _{woT}	32.00%	54.82%	13.33%	48.42%	11.52%	23.47%	6.06%	25.04%	16.00%	36.93%
	<i>Specine</i> _{wTF}	29.67%	53.25%	13.00%	48.03%	11.52%	24.80%	5.45%	24.44%	16.33%	36.63%
	<i>Specine</i> _{woA}	29.00%	52.02%	12.33%	45.93%	10.91%	22.15%	4.85%	23.83%	16.00%	35.71%
	<i>Specine</i> _{woAR}	23.33%	40.80%	10.67%	38.65%	9.70%	18.85%	4.24%	16.62%	14.00%	29.27%
	<i>Specine</i>	35.33%	59.62%	14.33%	50.99%	13.33%	28.67%	7.88%	30.16%	20.67%	45.58%
Qwen2.5-Coder	<i>Specine</i> _{woPTC}	31.33%	49.39%	13.33%	45.30%	12.73%	25.04%	9.09%	24.64%	16.00%	34.01%
	<i>Specine</i> _{woT}	33.33%	55.88%	15.00%	47.54%	13.94%	29.73%	9.09%	26.95%	15.67%	35.91%
	<i>Specine</i> _{wTF}	32.67%	55.87%	14.67%	49.33%	13.94%	30.12%	8.48%	27.45%	15.00%	33.77%
	<i>Specine</i> _{woA}	32.00%	55.18%	13.67%	48.04%	13.33%	29.85%	7.88%	26.98%	14.33%	32.14%
	<i>Specine</i> _{woAR}	24.00%	47.33%	11.33%	44.13%	13.33%	29.65%	7.27%	27.08%	13.67%	34.06%
	<i>Specine</i>	37.67%	60.63%	16.67%	51.53%	15.15%	32.44%	10.30%	34.11%	18.67%	42.09%
GPT-4o-mini	<i>Specine</i> _{woPTC}	51.00%	62.31%	20.33%	56.34%	17.58%	28.58%	13.33%	30.02%	33.67%	46.98%
	<i>Specine</i> _{woT}	53.00%	68.74%	20.67%	59.95%	22.42%	39.00%	14.55%	34.27%	34.33%	49.37%
	<i>Specine</i> _{wTF}	52.00%	67.76%	21.00%	60.07%	21.82%	38.56%	15.15%	37.07%	35.00%	49.33%
	<i>Specine</i> _{woA}	50.33%	65.24%	20.33%	57.46%	20.61%	37.29%	13.33%	35.19%	34.00%	48.07%
	<i>Specine</i> _{woAR}	42.67%	56.53%	18.33%	52.06%	21.82%	38.44%	12.73%	34.28%	30.67%	46.18%
	<i>Specine</i>	56.33%	72.98%	22.67%	63.55%	23.64%	40.65%	17.58%	42.10%	38.67%	57.34%
Gemini-1.5-Flash	<i>Specine</i> _{woPTC}	59.67%	70.91%	24.67%	61.08%	41.21%	51.51%	33.33%	45.86%	37.00%	52.10%
	<i>Specine</i> _{woT}	61.67%	74.14%	27.67%	63.59%	43.03%	56.62%	34.55%	49.60%	34.00%	52.01%
	<i>Specine</i> _{wTF}	60.67%	71.72%	27.33%	60.78%	41.82%	53.93%	31.52%	43.64%	36.33%	52.32%
	<i>Specine</i> _{woA}	59.00%	70.22%	24.33%	59.35%	40.61%	53.71%	30.91%	43.08%	35.33%	51.93%
	<i>Specine</i> _{woAR}	54.33%	66.91%	23.00%	57.93%	38.79%	49.37%	29.70%	42.11%	31.00%	46.78%
	<i>Specine</i>	65.33%	78.15%	29.33%	65.91%	44.85%	56.96%	36.36%	54.34%	38.67%	58.42%

outperforms all 10 baselines, even those leveraging public test cases, with average improvements of 7.37%~48.25% and 2.17%~33.15% in terms of Pass@1 and AvgPassRatio, respectively. This highlights the generalizability of *Specine*, demonstrating its effectiveness even in practical settings where public test cases are unavailable.

Secondly, *Specine* demonstrates superior performance compared to *Specine*_{wTF}, achieving average improvements of 14.75% and 12.95% in terms of Pass@1 and AvgPassRatio, respectively. These results validate the contribution of the specification lifting component and further confirm that it outperforms the test-execution feedback strategy commonly employed by existing agent-based techniques. In the future, we aim to further explore the integration of our specification lifting strategy to enhance the effectiveness of existing agent-based code generation techniques.

Thirdly, *Specine* outperforms both *Specine*_{woA} and *Specine*_{woAR} with average improvements of 20.94% and 34.57% in terms of Pass@1, and 16.49% and 28.75% in terms of AvgPassRatio, respectively. These results confirm the necessity of the specification alignment component in *Specine*. Moreover, *Specine*_{woAR} demonstrates the weakest performance among all variants, further underscoring the significance of the ten alignment rules derived from software requirements engineering, which contribute the most substantially to the overall effectiveness of *Specine*.

Furthermore, the *Wilcoxon Signed-Rank Test* [55] at a significance level of 0.05 confirms that all p-values are smaller than 1.77×10^{-7} , exhibiting the statistically significant advantage of *Specine* over all variants. Overall, each of the main components contributes substantially to the overall effectiveness of *Specine*.

6 Discussion

6.1 Case Study

During the alignment process, *Specine* outputs intermediate aligned specifications, enabling systematic analysis of its iterative alignment process. We present a case study using CodeContests benchmark and Gemini-1.5 (shown in Figure 5). The quality of the aligned specification is quantitatively evaluated based on the code correctness with private test cases: (1) Initially, the LLM generates code solely from the input specification, achieving a 0% pass ratio, indicating that the LLM fails to correctly perceive the specification. (2) *Specine* leverages the *specification purpose* rule to generate an aligned specification, improving the pass ratio to 77.56%, demonstrating that this alignment rule enhances the LLM’s perception. Consequently, this alignment rule is retained. (3) In the next iteration, *Specine* incorporates the *input requirements* rule, increasing the pass ratio to 80.00%, confirming its contribution. (4) Subsequent iterations yield no further improvements, leading to the removal of ineffective alignment rules. (5) By the sixth iteration, *Specine* incorporates the *hints or tips* rule, achieving a 100.00% pass ratio, indicating that the LLM fully perceives the final aligned specification. This provides a clear illustration of how *Specine*’s iterative alignment mechanism enhances the LLM perception, ultimately improving code generation performance.

To better understand the limitations of *Specine*, we conduct a manual analysis of 60 representative failure cases, sampled across four LLMs and five benchmarks. The analysis reveals that most failures are attributed to (1) suboptimal test cases generated by the tester agent (23 cases) and (2) incorrect application of alignment

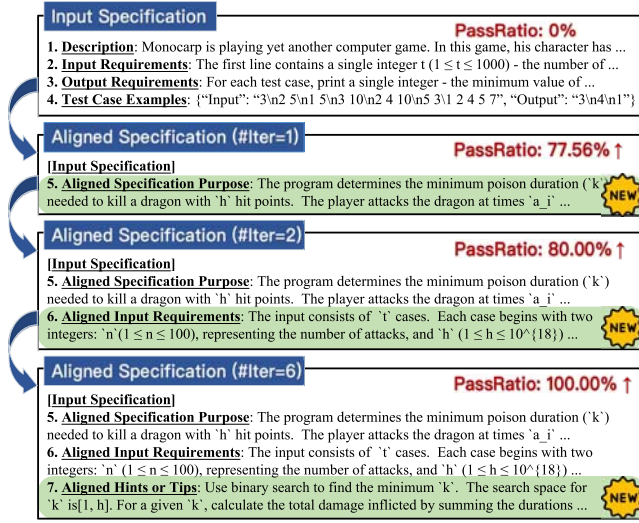


Figure 5: Case study on CodeContests with Gemini-1.5

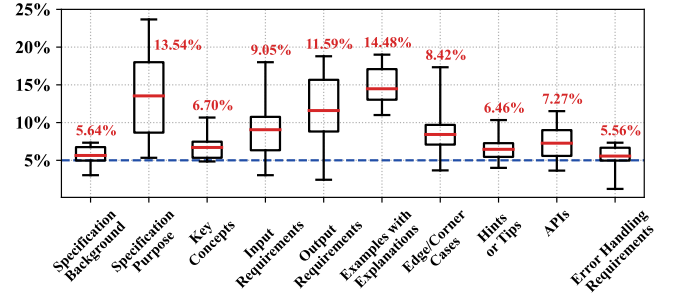
rules by the aligner agent (25 cases). A smaller subset (12 cases) involves inaccurate lifted specifications produced by the lifter agent. These findings highlight opportunities for further enhancement. In future work, we plan to enhance the robustness of the tester agent by incorporating specification alignment into the test generation process and introducing test case validation mechanisms. For the aligner agent, we intend to develop more effective alignment rules to mitigate specification misalignment.

6.2 Influence of Alignment Rules

We further analyze the influence of each alignment rule (as defined in Section 3.3). Specifically, for each programming problem, if a specific alignment rule leads to an improvement in the test pass ratio of generated code, it is considered an effective alignment rule for that problem. Based on this criterion, we evaluate the contribution of each alignment rule across all programming problems, considering all four LLMs and five benchmarks (as illustrated Figure 6). The results demonstrate that, on average, all alignment rules contribute to aligning more than 5% of all programming problems, further strengthening their necessity. Additionally, we observe that the three most effective alignment rules are *Examples with Explanations*, *Specification Purpose*, and *Output Requirements*, which successfully align an average of 14.48%, 13.54%, and 11.59% of all programming problems, respectively. This finding suggests that developers should focus on improving these three aspects when writing programming specifications for LLMs in code generation. In the future, we plan to design additional alignment rules (e.g., performance, security, and deployment requirements) to further enhance *Specine*'s effectiveness in handling more complex problems.

6.3 Quality of LLM-Generated Test Cases

The quality of LLM-generated test cases directly impacts the effectiveness of *Specine*. To comprehensively evaluate test case quality, we verify their correctness against the canonical solutions provided in the dataset follow existing research [25]. That is, an LLM-generated test case is considered correct if it passes the canonical

Figure 6: Effectiveness of each alignment rule in *Specine*

solution. Specifically, the accuracy of LLM-generated test cases is 60.17%~72.92% on average across different LLMs and benchmarks, indicating that a non-negligible proportion of LLM-generated test cases contain errors. We acknowledge that the presence of incorrectly generated test cases is a general challenge for all LLM-based approaches, primarily due to inherent limitations in LLM performance. Moreover, the accuracy of the generated test cases exceeds 50%, with the proportion of correct test cases surpassing that of incorrect ones. Therefore, this facilitates prioritization of correct code, as it exhibits a higher pass ratio on LLM-generated test cases than incorrect code, aligning with the voting principle. Our ablation study on *Specine*_{woPTC} further confirms that LLM-generated test cases significantly enhance the overall effectiveness of *Specine*, providing empirical evidence of their contribution. Similar findings have been reported in CodeT [5] and FlowGen [34]. To further relieve this challenge, we introduce a hierarchical pass ratio criterion (in Section 3.3) that prioritizes the results of reliable public test cases. This helps mitigate the negative impact of erroneous LLM-generated test cases to some extent. In future work, we aim to develop more advanced test case generation techniques, such as incorporating type checking and test coverage to guide LLM-based test generation, further improving the effectiveness of *Specine*.

6.4 Scalability of DSL

While our current evaluation focuses on structured programming tasks, the DSL design in our *Specine* is designed to be modular and extensible. The core principle of *Specine*, which involves abstracting LLM-generated code into a structured and semantically rich representation to improve alignment with user intent, is broadly applicable beyond competitive programming. The current schema builds on established foundations in requirements engineering [18, 19] and aligns with *IEEE specification standard* [11], ensuring wide applicability across diverse code generation scenarios. It can be extended to incorporate domain-specific ingredients (e.g., UI layout constraints in front-end development, data schemas in analytics) without modifying the overall alignment pipeline. In future work, we plan to investigate the use of advanced LLMs to generate DSL schemas tailored to new domains, with refinement via expert knowledge.

7 Threats and Validity

The threat to *construct* validity mainly lies in the inherent randomness involved in LLMs. On one hand, we conduct a large-scale study, and the consistency of our findings across all subjects helps

mitigate this threat. On the other hand, we repeat *Specine*'s experiments three times across all 20 subjects (4 LLMs $\times$ 5 benchmarks). Notably, the standard derivations are only 0.008 and 0.022 in terms of Pass@1 and AvgPassRatio, respectively, demonstrating the robustness of our conclusions to a large extent. Furthermore, a *Wilcoxon Signed-Rank Test* [55] at the significance level of 0.05 confirms that all p-values exceed 0.55, indicating no statistically significant differences across the three experimental results. This further strengthens the reliability of our results.

The threat to *external* validity mainly lies in the used subjects. To mitigate this, we carefully select a diverse set of benchmarks, metrics, baselines, and LLMs. Following prior studies [27, 29, 51], we utilize five widely-used benchmarks in code generation and employ two metrics to assess code correctness. Additionally, we compare *Specine* with ten popular or state-of-the-art baselines on four advanced LLMs, ensuring a comprehensive evaluation. In the future, we will extend our evaluation to a broader set of benchmarks and LLMs to further assess *Specine*'s generalizability.

8 Related Work

Code generation is a critical task in software engineering and has garnered significant attention in recent years [9, 32, 49, 59]. Among the various approaches, prompt-based and agent-based techniques are the two most widely-adopted paradigms.

Prompt-based techniques enhance code generation performance of LLMs by designing carefully-crafted prompts that can be applied in a plug-and-play manner. Based on their design principles and application stages, these techniques can be categorized into three types: (1) Pre-generation techniques (e.g., Self-planning [29] and SCoT [32]) aim to guide LLMs in generating intermediate reasoning steps, thereby improving the effectiveness of code generation. (2) Post-generation techniques (e.g., Self-debugging [8], Self-edit [60], and Self-repair [41]) leverage error messages from test execution to enable LLMs to refine and correct erroneous code. (3) Hybrid techniques (e.g., μ Fix [51]) integrate the advantages of both pre-generation and post-generation approaches, leveraging their synergy to further enhance code generation performance of LLMs.

Agent-based techniques improve the code generation performance of LLMs by simulating human collaborative software development processes, where multiple agents interact to accomplish tasks such as task planning, code generation, and testing. Each agent is assigned a specialized role, and their collaborative interactions optimize different stages of the code generation process. Representative agent-based techniques, including Self-collaboration [13], AgentCoder [25], and PairCoder [59] (introduced in Section 1 and 4.3), highlight the importance of role definition and multi-agent coordination in improving code generation performance of LLMs.

Unlike existing techniques, *Specine* introduces a novel perspective to enhance the code generation performance of LLMs through specification alignment. Our experimental results demonstrate that *Specine* significantly outperforms state-of-the-art prompt-based and agent-based code generation techniques in terms of effectiveness. Notably, *Specine* is orthogonal to existing prompt-based and agent-based techniques to some extent. Specifically, *Specine* can serve as a pre-processing module, aligning requirement specifications before they are provided to other techniques. In future work, we

plan to explore the integration of *Specine* with existing techniques to further enhance their performance.

9 Conclusion

In this work, we draw inspiration from software requirements engineering practices and propose a novel perspective for enhancing the code generation performance of LLMs through specification alignment. Based on this perspective, we design *Specine*, which consists of identifying misaligned input specifications, lifting LLM-perceived specifications, and aligning them to further generate the correct code solution. To evaluate the effectiveness of *Specine*, we conduct comprehensive experiments on four advanced LLMs and five challenging benchmarks. Experimental results demonstrate that *Specine* consistently outperforms state-of-the-art prompt-based and agent-based code generation techniques across multiple evaluation metrics, highlighting its superiority in enhancing the code generation performance of LLMs.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (Grant No. 62322208).

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Google AI. 2025. Gemini Developer API. <https://ai.google.dev/gemini-api/docs/>.
- [3] Anil Altinay, Joseph Nash, Taddeus Kroes, Prabhu Rajasekaran, Dixon Zhou, Adrian Dabrowski, David Gens, Yeoul Na, Stijn Volckaert, Cristiano Giuffrida, et al. 2020. BinRec: dynamic binary lifting and recompilation. In *Proceedings of the Fifteenth European Conference on Computer Systems*. 1–16.
- [4] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).
- [5] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2022. CodeT5: Code Generation with Generated Tests. In *The Eleventh International Conference on Learning Representations*.
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [7] Xiangping Chen, Xing Hu, Yuan Huang, He Jiang, Weixing Ji, Yanjie Jiang, Yanyan Jiang, Bo Liu, Hui Liu, Xiaochen Li, et al. 2025. Deep learning-based software engineering: progress, challenges, and opportunities. *Science China Information Sciences* 68, 1 (2025), 111102.
- [8] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024. Teaching Large Language Models to Self-Debug. In *The Twelfth International Conference on Learning Representations*.
- [9] Codeforces. 2025. <https://codeforces.com/>.
- [10] Google DeepMind. 2025. <https://deepmind.google/>.
- [11] John Doe. 2011. Recommended practice for software requirements specifications (ieee). *IEEE, New York* (2011).
- [12] Yihong Dong, Jiazhen Ding, Xue Jiang, Ge Li, Zhuo Li, and Zhi Jin. 2024. CodeScore: Evaluating Code Generation by Learning Code Execution. *ACM Trans. Softw. Eng. Methodol.* (2024).
- [13] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2024. Self-collaboration code generation via chatgpt. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–38.
- [14] EvalPlus. 2025. Code Sanitizer. <https://github.com/evalplus/evalplus/blob/master/evalplus/sanitize.py>.
- [15] EvalPlus. 2025. EvalPlus Leaderboard. <https://evalplus.github.io/leaderboard.html>.
- [16] Pengfei Gao, Zhao Tian, Xiangxin Meng, Xinchun Wang, Ruida Hu, Yuanan Xiao, Yizhou Liu, Zhao Zhang, Junjie Chen, Cuiyun Gao, et al. 2025. Trae Agent: An LLM-based Agent for Software Engineering with Test-time Scaling. *arXiv preprint arXiv:2507.23370* (2025).

- [17] Shuzheng Gao, Cuiyun Gao, Wenchao Gu, and Michael Lyu. 2025. Search-based llms for code optimization. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 254–266.
- [18] Martin Glinz. 2000. Problems and deficiencies of UML as a requirements specification language. In *Tenth International Workshop on Software Specification and Design. IWSSD-10 2000*. IEEE, 11–22.
- [19] Sol Greenspan, John Mylopoulos, and Alex Borgida. 1994. On formal requirements modeling languages: RML revisited. In *Proceedings of 16th International Conference on Software Engineering*. IEEE, 135–147.
- [20] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [21] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming—The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).
- [22] Qi Guo, Xiaofei Xie, Shangqing Liu, Ming Hu, Xiaohong Li, and Lei Bu. 2025. Intention is All You Need: Refining Your Code from Your Intention. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society.
- [23] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring Coding Challenge Competence With APPS. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*. <https://openreview.net/forum?id=sD93GOzH3i5>
- [24] Sirui Hong, Xiaowu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. 2023. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352* (2023).
- [25] Dong Huang, Qingwen Bu, Jie M Zhang, Michael Luck, and Heming Cui. 2023. Agentcoder: Multi-agent-based code generation with iterative testing and optimisation. *arXiv preprint arXiv:2312.13010* (2023).
- [26] Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).
- [27] Md. Ashrafur Islam, Mohammed Eunus Ali, and Md Rizwan Parvez. 2024. Map-Coder: Multi-Agent Code Generation for Competitive Problem Solving. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 4912–4944.
- [28] Naman Jain, Tianjun Zhang, Wei-Lin Chiang, Joseph E Gonzalez, Koushik Sen, and Ion Stoica. 2023. Improving code style for accurate code generation. In *NeurIPS 2023 Workshop on Synthetic Data Generation with Generative AI*.
- [29] Xue Jiang, Yihong Dong, Lecheng Wang, Zheng Fang, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. 2024. Self-planning code generation with large language models. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–30.
- [30] Majeed Kazemitabaar, Justin Chow, Carl Ka To Ma, Barbara J Ericson, David Weintrop, and Tovi Grossman. 2023. Studying the effect of AI code generators on supporting novice learners in introductory programming. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–23.
- [31] Mohammad Abdullah Matin Khan, M Saiful Bari, Xuan Long Do, Weishi Wang, Md Rizwan Parvez, and Shafiq Joty. 2024. XCodeEval: An Execution-based Large Scale Multilingual Multitask Benchmark for Code Understanding, Generation, Translation and Retrieval. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 6766–6805.
- [32] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2025. Structured chain-of-thought prompting for code generation. *ACM Transactions on Software Engineering and Methodology* 34, 2 (2025), 1–23.
- [33] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.
- [34] Feng Lin, Dong Jae Kim, and Tse-Hsun (Peter) Chen. 2025. SOEN-101: Code Generation by Emulating Software Process Models Using Large Language Model Agents. In *2025 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*.
- [35] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems* 36 (2023), 21558–21572.
- [36] Linda A Macaulay. 2012. *Requirements engineering*. Springer Science & Business Media.
- [37] Noble Saji Mathews and Meiyappan Nagappan. 2024. Test-Driven Development and LLM-based Code Generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1583–1594.
- [38] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binquan Zhang, ChenXue Wang, Shichao Liu, and Qing Wang. 2024. Clarifygpt: A framework for enhancing llm-based code generation via requirements clarification. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 2332–2354.
- [39] Nico Naus, Freek Verbeek, Sagar Atla, and Binoy Ravindran. 2024. Poster: Formally Verified Binary Lifting to P-Code. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 4973–4975.
- [40] Homepage of Specnie. 2025. <https://github.com/tianzhaotju/Specnie>.
- [41] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. 2024. Is Self-Repair a Silver Bullet for Code Generation?. In *The Twelfth International Conference on Learning Representations*.
- [42] OpenAI. 2024. Introducing GPT-4o and more tools to ChatGPT free users. (2024).
- [43] OpenAI. 2025. The most powerful platform for building AI products. <https://openai.com/api/>.
- [44] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirel. 2023. The impact of ai on developer productivity: Evidence from github copilot. *arXiv preprint arXiv:2302.06590* (2023).
- [45] LeetCode The World's Leading Online Programming Learning Platform. 2025. <https://leetcode.com/>.
- [46] Klaus Pohl. 1996. *Requirements engineering: An overview*. Citeseer.
- [47] Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530* (2024).
- [48] Zhao Tian, Junjie Chen, and Zhi Jin. 2023. Code difference guided adversarial example generation for deep code models. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 850–862.
- [49] Zhao Tian, Junjie Chen, Dong Wang, Qihao Zhu, Xingyu Fan, and Lingming Zhang. [n. d.]. LEAM++: Learning for Selective Mutation Fault Construction. *ACM Transactions on Software Engineering and Methodology* ([n. d.]).
- [50] Zhao Tian, Junjie Chen, and Xiangyu Zhang. 2023. On-the-fly improving performance of deep code models via input denoising. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 560–572.
- [51] Zhao Tian, Junjie Chen, and Xiangyu Zhang. 2025. Fixing Large Language Models' Specification Misunderstanding for Better Code Generation. In *2025 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*.
- [52] Zhao Tian, Junjie Chen, Qihao Zhu, Junjie Yang, and Lingming Zhang. 2022. Learning to construct better mutation faults. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.
- [53] Zhao Tian, Honglin Shu, Dong Wang, Xuejie Cao, Yasutaka Kamei, and Junjie Chen. 2024. Large language models for equivalent mutant detection: How far are we?. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1733–1745.
- [54] Bingyang Wei. 2024. Requirements are All You Need: From Requirements to Code with LLMs. *arXiv preprint arXiv:2406.10101* (2024).
- [55] Frank Wilcoxon, S Katti, Roberta A Wilcox, et al. 1970. Critical values and probability levels for the Wilcoxon rank sum test and the Wilcoxon signed rank test. *Selected tables in mathematical statistics* 1 (1970), 171–259.
- [56] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2019. Huggingface's transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771* (2019).
- [57] Zhen Yang, Fang Liu, Zhongxing Yu, Jacky Wai Keung, Jia Li, Shuo Liu, Yifan Hong, Xiaoxue Ma, Zhi Jin, and Ge Li. 2024. Exploring and unleashing the power of large language models in automated code translation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1585–1608.
- [58] Pamela Zave and Michael Jackson. 1997. Four dark corners of requirements engineering. *ACM transactions on Software Engineering and Methodology (TOSEM)* 6, 1 (1997), 1–30.
- [59] Huan Zhang, Wei Cheng, Yuhang Wu, and Wei Hu. 2024. A Pair Programming Framework for Code Generation via Multi-Plan Exploration and Feedback-Driven Refinement. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1319–1331.
- [60] Kechi Zhang, Zhuo Li, Jia Li, Ge Li, and Zhi Jin. 2023. Self-Edit: Fault-Aware Code Editor for Code Generation. *arXiv preprint arXiv:2305.04087* (2023).
- [61] Naifeng Zhang, Sanil Rao, Mike Fransusich, and Franz Franchetti. 2025. Towards Semantics Lifting for Scientific Computing: A Case Study on FFT. *arXiv preprint arXiv:2501.09201* (2025).
- [62] Shun Zhang, Zhenfang Chen, Yikang Shen, Mingyu Ding, Joshua B Tenenbaum, and Chuang Gan. 2023. Planning with Large Language Models for Code Generation. In *The Eleventh International Conference on Learning Representations*.